

U3 · Aula 2 — Playwright: Visual Regression, Trace Viewer e CI com Sharding

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · Qualidade Web e Mobile (EAD)

Roteiro da aula

- **Parte 1 — Visual e debug:** visual regression, trace viewer, component testing
- **Parte 2 — CI em escala:** sharding, blob reports, quality gate
- **Parte 3 — Prática guiada:** snapshots do CineFav Web + CI no seu fork

Depois desta aula: screencast — eu resolvendo os specs passo a passo.

Objetivos de aprendizagem

- **Implementar** visual regression com `toHaveScreenshot` + baseline versionado
- **Diagnosticar** falhas remotas com trace viewer (sem reproduzir local)
- **Paralelizar** a suíte com sharding e consolidar com blob reports
- **Calibrar** tolerância visual (threshold, maxDiffPixels, mask)
- **Gerar** os baselines do CineFav Web em 3 viewports

Parte 1 — Visual e debug

toHaveScreenshot · trace viewer · component testing

Visual regression nativo

```
test('home page visual', async ({ page }) => {  
  await page.goto('/');  
  
  await expect(page).toHaveScreenshot('home.png', {  
    maxDiffPixels: 100,           // tolera 100px diferentes  
    threshold: 0.2,              // tolerância perceptual 0-1  
    animations: 'disabled',      // congela animações  
  });  
});
```

```
# Primeira execução: gera baseline  
npx playwright test --update-snapshots  
  
# Execuções seguintes: compara contra baseline  
npx playwright test
```

Baseline fica versionado no repo — revisão de mudança visual explícita no PR.

Visual regression multi-viewport

```
// playwright.config.ts
export default defineConfig({
  projects: [
    { name: 'desktop', use: { viewport: { width: 1280, height: 720 } } },
    { name: 'tablet', use: { viewport: { width: 768, height: 1024 } } },
    { name: 'mobile', use: { viewport: { width: 390, height: 844 } } },
  ],
});
```

```
// O mesmo teste roda nos 3 viewports automaticamente
test('checkout visual', async ({ page }) => {
  await page.goto('/checkout');
  await expect(page).toHaveScreenshot(); // nome gerado por projeto
});
```

Trace viewer — debug sem reproduzir local

```
// playwright.config.ts
use: {
  trace: 'retain-on-failure', // captura trace só quando falha
  // trace: 'on'             // captura sempre (pesado)
}
```

```
# Abrir trace de uma falha
npx playwright show-trace test-results/.../trace.zip
```

O trace viewer mostra:

- Timeline de cada ação
- Snapshots do DOM antes/depois de cada passo
- Network requests e responses
- Console logs
- Screenshot do momento da falha

Component testing

Alternativa ao Cypress Component Testing.

```
// LoginForm.spec.tsx
import { test, expect } from '@playwright/experimental-ct-react';
import LoginForm from './LoginForm';

test('desabilita submit com email inválido', async ({ mount }) => {
  const component = await mount(<LoginForm />);
  await component.getByLabel('Email').fill('invalido');
  await expect(component.getByRole('button', { name: 'Entrar' })).toBeDisabled();
});
```

Roda componente isolado em browser real — sem subir app inteiro. Mais rápido que E2E, mais realista que JSDOM.

Parte 2 — CI em escala

Sharding · blob reports · quality gate

Sharding — CI paralelo

```
# .github/workflows/playwright.yml
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
    matrix:
      shard: [1/4, 2/4, 3/4, 4/4] # 4 máquinas paralelas
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test --shard=${{ matrix.shard }}
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: blob-report-${{ matrix.shard }}
          path: blob-report
```

Blob reports — relatório consolidado

```
merge-reports:
  needs: test
  runs-on: ubuntu-latest
  steps:
    - uses: actions/download-artifact@v4
      with: { path: all-blob-reports, pattern: blob-report-* }

    - run: npx playwright merge-reports --reporter html ./all-blob-reports

    - uses: actions/upload-artifact@v4
      with:
        name: html-report
        path: playwright-report
```

1 relatório HTML unificado com resultados dos 4 shards. Clique num teste → abre trace.

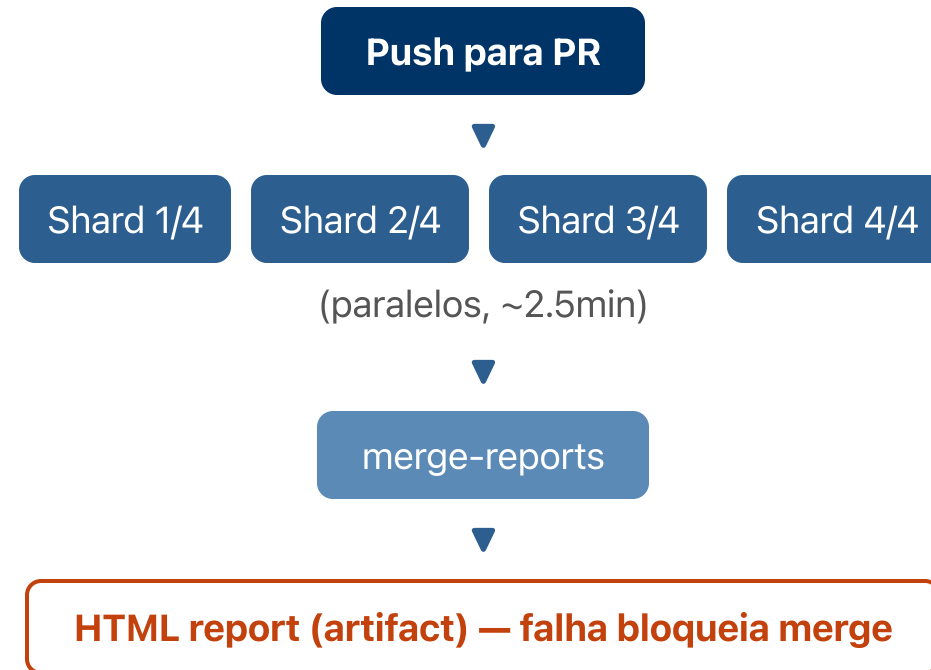
Ganho real de sharding

Suíte de exemplo: 200 testes × 3s cada = 600s (10min)

Configuração	Tempo
Sem sharding (1 máquina)	~10 min
2 shards	~5 min
4 shards	~2.5 min
8 shards	~1.25 min

GitHub Actions: cada shard = job separado com billing individual. Balance cost vs speed.

CI completo — visão geral



Pitfalls comuns

- ✗ **Baseline não versionado no repo** — diff incontrolável entre máquinas
- ✗ **threshold: 0** no visual — falha por anti-aliasing (sub-pixel rendering)
- ✗ **trace: 'on' sempre** — artefatos pesados, CI lento
- ✗ **CI sem sharding** em suítes grandes — feedback de hora(s)
- ✗ **Component test que mocka tudo** — perde o ponto de testar em browser real

Resumo

- **toHaveScreenshot** → visual regression nativo, baseline versionado no git
- **Trace viewer** → debug completo sem reproduzir local (dom + network + timeline)
- **Component testing** → velocidade de unit + realismo de browser
- **Sharding** → CI paralelo linear; **blob reports** → 1 relatório consolidado

Parte 3 — Prática guiada

Snapshots do CineFav Web + CI no seu fork

A prática — visual regression no CineFav Web

Mesmo app da aula anterior (`exercicios/02-lab-web-pwa-playwright/pratica`):

```
cd exercicios/02-lab-web-pwa-playwright/pratica
ls tests/e2e      # confirma: 03-visual.spec.ts está aí
```

- `03-visual.spec.ts` — scaffold com TODOs
- Prática 1 (resolvida no screencast): snapshot da tela de login
- Prática 2 (● desafio): home em **3 viewports** (mobile, tablet, desktop)
- Prática 3 (● desafio): **mask** na região dinâmica (botão de favorito)

O ciclo do baseline

```
# 1ª vez – gera os baselines na SUA máquina:  
npm run test:visual:update  
# → cria tests/e2e/03-visual.spec.ts-snapshots/*.png  
  
# depois – compara contra o baseline:  
npm run test:visual  
  
# mudou a UI de propósito? regenere e commite o novo baseline:  
npm run test:visual:update
```

Baseline **commitado no repo** = mudança visual vira diff revisável no PR.

É o critério "baseline versionado" da rubrica do lab.

Prática CI — sharding no seu fork

O repo já traz `.github/workflows/playwright.yml`:

- **2 shards** rodando em paralelo + `merge-reports` com blob reports
- Habilite o **Actions** na aba do seu fork → push → veja os 2 jobs
- Visual fica **fora do CI** (`--grep-invert "Visual"`): baseline local $\neq$ render do `ubuntu-latest` — gerar baseline **no CI** é o jeito certo (desafio bônus)

| Link da run verde no PR = parte da entrega do lab.

Como acompanhar

- **Screencast** — vídeo na sequência desta unidade
- **Diff visual falhou?** — `npx playwright show-report` mostra expected × actual × diff
- **Falha só no CI?** — baixe o trace no artifact e abra com `show-trace`

Bons estudos!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith